

LibTorch: Day 7

→ Building Neural Networks with
torch::nn

The torch::nn Module System

→ Like cars have pre-built parts, Neural Networks are the same. LibTorch gives you pre-built layers like `nn::Linear` (a mathematical layer), `nn::Conv2d` (a vision layer), `nn::LSTM` (a memory layer). You snap these together like LEGOs.

Key Pre-Built Layers

<u>Layer</u>	<u>What It Does</u>	<u>Common Use</u>
<code>nn::Linear(in, out)</code>	Fully connected layer. Most basic building block.	MLP, Classification heads
<code>nn::Conv2d(in, out, k)</code>	2D convolution. Detects spatial patterns.	Image recognition, object detection
<code>nn::Conv1d(in, out, k)</code>	1D convolution. Detects patterns in sequences	Time series, audio
<code>nn::LSTM(in, hidden, layers)</code>	Long Short Term Memory	NLP, Time series forecasting

similar layers available for $\rightarrow$

GRU (in. hidden, layers)

BatchNorm2d (features)

Dropout(p)

Embedding (vocab, dim)

ReLU()

Sigmoid()

Softmax(dim)

MaxPool2d(k)

AdaptiveAvgPool2d

* We can try building a Sequential Model (Simple, Linear Stack)

```
#include <torch/torch.h>
```

```
#include <iostream>
```

```
int main() {
```

```
    // Building a simple 3-layer MLP (Multi-Layer  
    Perceptron)
```

```
    // Input : 784 features
```

```
    // Output : 10 classes (digits 0-9)
```

```
    auto net = torch::nn::Sequential(
```

```
        torch::nn::Linear(784, 256),
```

```
        // Layer 1: 784  $\rightarrow$   
        256
```

```
        torch::nn::ReLU(),
```

```
        // Activation
```

```
        torch::nn::Dropout(0.3),
```

```
        // Dropout 30%
```

```
        torch::nn::Linear(256, 128),
```

```
        // Layer 2: 256  $\rightarrow$   
        128
```



```
torch::nn::ReLU(),  
torch::nn::Dropout(0.2),  
torch::nn::Linear(128, 10)
```

↳ // layer 3:
128 → 10

```
);
```

// Forward pass with dummy data

```
auto input = torch::randn({32, 784});
```

↓
Batch of 32 images

```
auto output = torch::net → forward(input);
```

```
std::cout << "Output shape: " << output.sizes()  
<< std::endl; // {32, 10}
```

// Count parameters

```
long total_params = 0;
```

```
for (auto& p : net → parameters()) {
```

```
    total_params += p.numel();
```

```
}
```

```
std::cout << "Total parameters: " << total_params  
<< std::endl;
```

```
return 0;
```

```
}
```